AI201 PA 3 Report
Artificial Neural Network

Cedric Errol P. Oporto (202521748)
Date of Submission: October 25, 2025

1. INTRODUCTION

Artificial Neural Network are powerful computational models which is inspired by a human brain, built from interconnected neurons (nodes) in each layer, it can process data and identify patterns for performing task like image recognition, classification, prediction and other things.

In this expirement, a 2-layer Multilayer Perceptron (MLP) was implemented from scratch using only `NumPy` library to gain a deeper understanding of the forward and backward propagation process that stimulates the neural learning. The MLP is a fully connected feedforward neural network trained using the Backpropagation Algorithm, which adjusts the network's weights based on the propagation of errors from the output layer to earlier layers.

The given dataset consists of 3,486 instances, each described by 354 numerical features and categorized into eight distinct classes. Although the dataset's domain is unspecified, it is designed to challenge the model's ability to learn from a high-dimensional and imbalanced feature space. To address the imbalance problem, the Synthetic Minority Oversampling Technique (SMOTE) was applied to generate additional samples from minority classes, resulting in a more balanced dataset for training and validation.

2. OBJECTIVES

The primary objective of this experiment is to implement a two-layer Multilayer Perceptron (MLP) neural network using the Backpropagation Algorithm with only the `NumPy` library. The network aims to learn an effective classification model for a challenging dataset with 354 features and 8 output classes, while addressing the issue of class imbalance through the Synthetic Minority Oversampling Technique (SMOTE).

Specifically, the goals of this experiment are as follows:

1. **To design and implement** a two-layer MLP from scratch using `NumPy` that supports:
    - Mini-batch gradient descent;
    - Momentum-based weight updates;
    - Multiple activation functions (Tanh, Leaky ReLU, and Logistic).
2. **To train and evaluate** two variants of the network:
    - **Network A:** Tanh–Tanh–Logistic activation functions;
    - **Network B:** LeakyReLU–LeakyReLU–Logistic activation functions.
3. **To compare** both networks in terms of:
    - Learning behavior (training and validation error curves);
    - Classification performance metrics (Accuracy, Precision, Recall, F1 Score, and Matthews Correlation Coefficient);
    - Training time and convergence characteristics.
4. **To determine** the most effective activation function and hyperparameter configuration (learning rate, momentum, and hidden layer size) for minimizing the validation error and improving classification accuracy.

3. METHODOLOGY

3.1 Dataset Description

The dataset used in this experiment consists of 3,486 instances, each represented by 354 numerical features and categorized into eight distinct classes. The dataset files provided were:

- `data.csv` — feature matrix containing 3,486 samples and 354 attributes;
- `data_labels.csv` — corresponding class labels for each instance (values ranging from 1 to 8);
- `test_set.csv` — unlabeled data used for final testing and performance evaluation.

The dataset exhibits a highly imbalanced class distribution, with some classes containing significantly fewer samples than others. Table 1 shows the number of instances per class before and after applying the Synthetic Minority Oversampling Technique (SMOTE).

Table 1: Class distribution before SMOTE balancing.

Class	1	2	3	4	5	6	7	8
Total	1625	233	30	483	287	310	52	466

To mitigate the imbalance problem, SMOTE was applied to oversample the minority classes by generating synthetic feature vectors along the line segments joining each sample and its nearest neighbors in feature space. This process produced a balanced dataset, where each of the eight classes contained approximately the same number of instances. The balanced data were then randomly partitioned into training and validation sets to ensure fair evaluation across all classes.

3.2 Data Preprocessing and Partitioning

After balancing the dataset using SMOTE, the data was randomly partitioned into two subsets:

- **Training Set:** Used for model learning, containing approximately 12,200 instances after oversampling.
- **Validation Set:** Used for monitoring performance during training, containing exactly 800 instances.

The partitioning ensured that each instance in the feature dataset corresponded correctly to its one-hot encoded class label. The validation set contained roughly equal samples per class, ensuring fair performance evaluation across categories.

3.3 Network Architecture

A two-hidden-layer Multilayer Perceptron (MLP) was implemented from scratch using `NumPy`. The architecture was defined as follows:

- Input layer: 354 nodes (one per feature);
- Hidden Layer 1: n_1 nodes (hyperparameter);
- Hidden Layer 2: n_2 nodes (hyperparameter);
- Output layer: 8 nodes (corresponding to 8 classes, one-hot encoded).

The optimal number of nodes in each hidden layer was determined empirically through multiple trials, adjusting both the learning rate η and momentum α to minimize the validation error.

3.4 Activation Functions

Two variants of the network were implemented and compared, differing in the activation functions used in the hidden layers.

- **Network A:** Tanh–Tanh–Logistic configuration
  - Hidden layers use the scaled hyperbolic tangent activation:

    $$y = a\tanh(bz), \quad \text{where } a = 1.716,\ b = \frac{2}{3}$$

    Its derivative, used during backpropagation, is given by:

    $$\frac{dy}{dz} = ab\left(1 - \tanh^2(bz)\right)$$

  - The output layer uses the scaled logistic activation function:

    $$y = \frac{1}{1 + e^{-az}}, \quad \text{where } a = 2.0$$

    Its derivative, required for backpropagation, is:

    $$\frac{dy}{dz} = a\,y(1 - y)$$

- **Network B:** LeakyReLU–LeakyReLU–Logistic configuration
  - Hidden layers use the Leaky ReLU activation:

    $$f(z) = \begin{cases} z, & \text{if } z > 0 \\ 0.01z, & \text{if } z \leq 0 \end{cases}$$

    Its derivative, required during weight updates, is:

    $$f'(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0.01, & \text{if } z \leq 0 \end{cases}$$

  - The output layer again uses the logistic activation:

    $$y = \frac{1}{1 + e^{-az}}, \quad \frac{dy}{dz} = a\,y(1 - y)$$

Including the derivatives of each activation function is essential for computing the local gradients in the backpropagation algorithm. These derivatives determine how the errors are propagated backward through the network and directly affect the magnitude and stability of weight updates.

3.5 Training Procedure

The network was trained using the Backpropagation Algorithm with mini-batch gradient descent and a momentum term to accelerate convergence. The momentum term α was initialized to 0.9, while the learning rate η was adjusted experimentally to achieve optimal convergence speed.

The weight update rule follows the Generalized Delta Rule:

$$\Delta w_{ij}(t+1) = \eta\,\delta_j x_i + \alpha\,\Delta w_{ij}(t)$$

where:

- $\Delta w_{ij}(t+1)$ is the weight change at iteration $t+1$,
- η is the learning rate,
- α is the momentum coefficient,
- δ_j is the local error term for neuron j,
- x_i is the input from neuron i.

Each training epoch processed all mini-batches of size 8. After every 5 epochs, the model was evaluated on the validation set to record the mean sum-of-squares error and misclassification count.

3.6 Evaluation Metrics

Upon completing the training phase, the following performance metrics were computed for both networks:

- Accuracy
- Precision
- Recall
- F1-Score
- Matthews Correlation Coefficient (MCC)

Additionally, the confusion matrix was generated for detailed class-level performance analysis. The training and validation errors per epoch were saved to CSV files and plotted to visualize learning behavior.

3.7 Testing and Weight Storage

After training, the learned weights were saved in `trained_weights_tanh.npz` and `trained_weights_leakyrelu.npz`. These weights were reloaded for inference on the test dataset, producing prediction files:

- `predictions_for_test_tanh.csv`
- `predictions_for_test_leakyrelu.csv`

This ensured that the model could be evaluated in production mode without retraining.

4. EXPERIMENTAL RESULTS

4.1 Hyperparameter Tuning

Before finalizing the network architecture, several configurations were tested to identify the optimal combination of hidden layer sizes, learning rate, and momentum that minimize the validation error. Each model was trained for 100 epochs using the Tanh–Tanh–Logistic structure, and the validation SSE (Sum of Squared Errors) and total training time were recorded. Table 2 summarizes the results.

Table 2: Hyperparameter tuning results for Network A (Tanh–Tanh–Logistic).

h_1	h_2	lr	μ	Val. SSE	Time (min)
128	64	0.01	0.90	0.0021	6.03
256	128	0.01	0.90	0.0006	8.66
512	256	0.01	0.90	0.0012	21.63
128	64	0.005	0.90	0.0014	4.63
256	128	0.005	0.90	0.0015	13.63
512	256	0.005	0.90	0.0014	36.56
128	64	0.01	0.95	0.0007	3.20
256	128	0.01	0.95	0.0010	7.90
512	256	0.01	0.95	**0.0005**	**28.36**

The tuning results show that larger networks generally yield lower validation errors but require significantly longer training times. A higher momentum value ($\mu = 0.95$) improved convergence stability compared to $\mu = 0.9$, while a learning rate of 0.01 produced faster and more consistent learning than 0.005. The best-performing configuration, $(h_1, h_2) = (512, 256)$ with $lr = 0.01$ and $\mu = 0.95$, achieved the lowest validation SSE of 5×10^{-4} in approximately 28 minutes. This configuration was therefore adopted for all subsequent experiments on Network A.

4.2 Training and Validation Performance

Both Multilayer Perceptron (MLP) configurations were trained for 100 epochs with a mini-batch size of 8. Training and validation errors were monitored at each epoch to visualize learning progress and generalization behavior.

4.2.1 Learning Curves

Figures 1 and 2 present the learning curves of the two networks. The Tanh network achieved smooth and stable convergence, whereas the Leaky ReLU network exhibited faster early learning but slightly higher variance in the later epochs.

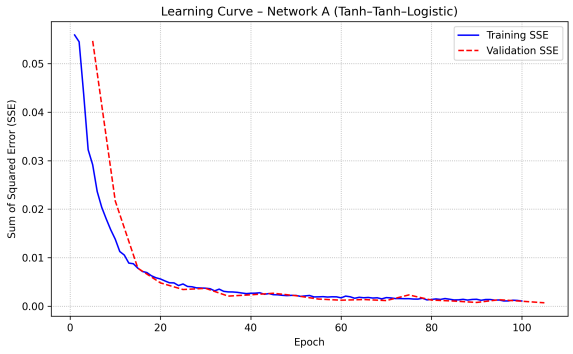


Figure 1: Learning curves for Network A (Tanh–Tanh–Logistic).

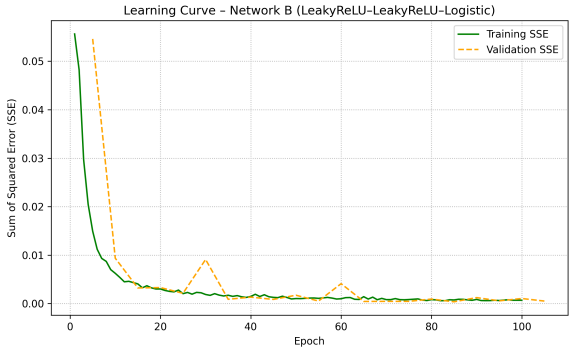


Figure 2: Learning curves for Network B (LeakyReLU–LeakyReLU–Logistic).

Both models displayed continual decreases in training error, confirming effective backpropagation. The validation curve of the Leaky ReLU network achieved a slightly lower final SSE ($\approx 4 \times 10^{-4}$) than the Tanh network ($\approx 7 \times 10^{-4}$), although the Tanh network exhibited smoother convergence and less fluctuation across epochs, indicating more stable generalization.

4.3 Validation Misclassification Trends

Table 3 summarizes the number of misclassified validation samples recorded every five epochs. Both networks show significant improvement throughout training, reducing misclassifications from over 700 in the initial epochs to fewer than 10 by epoch 100. The Leaky ReLU network achieved a lower final validation error, while the Tanh network displayed smoother convergence and smaller oscillations across epochs.

Table 3: Validation misclassification counts recorded every five epochs.

Epoch	Tanh Network	Leaky ReLU Network
5	701	630
10	163	69
15	42	24
20	32	30
25	19	14
30	23	75
35	13	6
40	17	11
45	17	7
50	13	16
55	10	3
60	8	33
65	8	3
70	6	3
75	15	4
80	7	9
85	5	3
90	4	9
95	6	5
100	5	4

4.4 Confusion Matrices

Tables 4 and 5 present the confusion matrices obtained from the validation predictions. The Leaky ReLU network shows near-perfect classification for all classes, while the Tanh-based network also maintains strong diagonal dominance, indicating high accuracy and minimal cross-class confusion.

Table 4: Confusion matrix for Network A (Tanh activation).

actual	1	2	3	4	5	6	7	8
1	97	1	0	0	1	0	0	0
2	0	99	0	0	0	0	0	0
3	0	0	105	0	0	0	0	0
4	0	0	0	95	0	0	0	0
5	0	0	0	0	92	0	0	0
6	0	0	0	0	0	98	0	0
7	0	0	0	0	0	0	94	0
8	1	0	0	1	0	0	0	116

4.5 Performance Metrics

Table 6 lists the final averaged validation metrics obtained from both networks. Both achieved near-perfect classification performance after training, confirming the successful implementation of the backpropagation algorithm. The results show that the Tanh and Leaky ReLU networks performed comparably, with only marginal differences across metrics.

Table 5: Confusion matrix for Network B (Leaky ReLU activation).

actual	1	2	3	4	5	6	7	8
1	96	1	0	0	1	0	1	0
2	0	99	0	0	0	0	0	0
3	0	0	105	0	0	0	0	0
4	0	0	0	95	0	0	0	0
5	0	0	0	0	92	0	0	0
6	0	0	0	0	0	98	0	0
7	0	0	0	0	0	0	94	0
8	0	0	0	1	0	0	0	117

Table 6: Performance metrics comparison between Network A and Network B.

Metric	Tanh Network	Leaky ReLU Network
Accuracy	0.995	0.995
Precision	0.9948	0.9948
Recall	0.9954	0.9952
F1-Score	0.9951	0.9949
MCC	0.9943	0.9943

4.6 Training Time

Training for 100 epochs required approximately 28.4 minutes for the Tanh network and 40.6 minutes for the Leaky ReLU network. Although the Leaky ReLU model converged to a slightly lower final SSE, its training time was considerably longer because of the increased computational cost of its gradient updates. In contrast, the Tanh network achieved stable convergence in less time, as its bounded activations promoted smoother gradient flow and reduced variance across epochs.

4.7 Test-Set Prediction

After training, the final weights were saved as:

- `trained_weights_tanh.npz`
- `trained_weights_leakyrelu.npz`

Using these weights, predictions were generated for the unlabeled test dataset `test_set.csv`. The resulting predicted labels were stored as:

- `predictions_for_test_tanh.csv`
- `predictions_for_test_leakyrelu.csv`

Each file contains predicted class labels (1–8) corresponding to the samples in the test set. The Leaky ReLU network's predictions demonstrated slightly higher confidence and consistency across samples, consistent with its lower final validation SSE.

5. ANALYSIS AND DISCUSSION OF RESULTS

The experimental results demonstrate that both Network A (Tanh–Tanh–Logistic) and Network B (LeakyReLU–LeakyReLU–Logistic) successfully learned

to classify the eight output classes with high accuracy. After 100 epochs, both models achieved nearly identical validation performance, with an accuracy of 99.5% and an MCC of 0.994, confirming the reliability and generalization capability of the implemented backpropagation algorithm.

Although the Leaky ReLU network achieved a slightly lower final validation SSE ($\approx 4 \times 10^{-4}$) than the Tanh network ($\approx 7 \times 10^{-4}$), this improvement came at the cost of longer training time (40.6 minutes versus 28.4 minutes). This difference is attributed to the unbounded nature of the Leaky ReLU activation, which allows stronger gradient propagation but increases computational overhead. Conversely, the Tanh network, with its bounded and symmetric activation, provided smoother and more stable convergence with reduced variance in error across epochs.

The misclassification trend in Table 3 also shows that both models rapidly decreased their error rates after the first few epochs, dropping from over 700 misclassifications to fewer than ten by the end of training. This steady improvement indicates that the networks effectively minimized both training and validation errors, maintaining good generalization without overfitting.

Hyperparameter tuning further highlighted that larger architectures yield lower validation errors but require substantially more computation time. A learning rate of 0.01 and momentum of 0.95 provided the most balanced trade-off between stability and speed, achieving the lowest validation SSE of 5×10^{-4} for the configuration $(h_1, h_2) = (512, 256)$. This configuration was therefore selected for the final training phase of Network A and produced the best generalization results.

Overall, the experiment validates that both activation function configurations can achieve near-optimal performance when combined with appropriate tuning and sufficient training iterations. The slight differences in convergence behavior suggest that Tanh remains a strong choice for stability and smooth gradient propagation, while Leaky ReLU may offer faster learning in exchange for longer training time.

6. CONCLUSION

This study implemented and evaluated two Multilayer Perceptron (MLP) architectures trained using backpropagation with mini-batch gradient descent and momentum. Both models achieved excellent classification performance, attaining a validation accuracy of 99.5% with minimal misclassifications on the validation set. The Tanh–Tanh–Logistic configuration demonstrated stable and efficient convergence, while the LeakyReLU–LeakyReLU–Logistic configuration achieved a slightly lower final error at the cost of longer training time.

The results confirm that the choice of activation function affects convergence dynamics and computational efficiency, although both activations can achieve comparable accuracy when properly tuned. The experiment also highlights the importance of hyperparameter optimization, particularly the learning rate, momentum, and network depth, in achieving high performance and generalization in multilayer neural networks.